Final Project

Reproducing and Extending Classical Reinforcement Learning Papers

Each team chooses one of the three tracks below. The intent is not to reproduce the original paper at full industrial scale. Instead, teams should reproduce a scaled but faithful benchmark, then adapt the method to a related environment that was not the original headline testbed and that forces a meaningful algorithmic change.

> Reproduction shows that you understand the paper; adaptation shows that you can reason about why a method breaks and how to change it.

Project choices at a glance

Track	Canonical paper	Reproduction target	Adaptation target	Main required change
DQN	Mnih et al. (2015)	ALE Breakout or Pong	MiniGrid MemoryEnv	Add explicit memory (recurrent DQN or equivalent)
PPO	Schulman et al. (2017)	HalfCheetah-v5 or Hopper-v5	MPE2 Simple Spread	Extend PPO to multi-agent coordination
AlphaZero	Silver et al. (2017/2018)	OpenSpiel Connect Four	OpenSpiel Backgammon	Handle chance nodes in search and training

Common requirements for all teams

- Read the paper closely enough to identify the original objective, training loop, state or action representation, and evaluation protocol.
- Use the official environment families linked in this document unless the instructor approves a close substitute in advance.
- Separate the project into two phases: (1) canonical reproduction and (2) adaptation to a new environment.
- The adaptation must include at least one non-trivial algorithmic or architectural change. Pure hyperparameter tuning, reward rescaling alone, or switching libraries is not enough.
- Run at least 3 random seeds for the main reported results.
- Track training reward or return and at least one task-specific metric such as success rate, collision count, or win rate.
- Save enough information to reproduce the final plots: code version, configs, seed values, and the hyperparameters used for the final runs.
- Students may use existing wrappers, logging tools, and environment utilities, but the core learning algorithm and the adaptation should be implemented or substantially modified by the team and clearly explained.

Suggested deliverables

- A code repository with a README and exact setup instructions.
- A short report (recommended length: 6-8 pages) explaining the paper, the reproduction setup, the adaptation, the evaluation, and the main lessons learned.
- A final presentation or poster with the key curves and one short qualitative demo (for example, gameplay clips, trajectories, or rollout screenshots).
- A compact appendix or table listing the final hyperparameters, environment versions, and compute budget.

Minimum evaluation evidence

- One plot or table for the canonical reproduction task.

- One direct comparison showing how the unmodified method behaves on the new environment.
- One comparison showing the adapted method on that same environment.
- At least one ablation isolating the main design change.
- A brief discussion of where the reproduction diverged from the original paper and why.

Track 1: DQN

Classical value-based deep reinforcement learning from pixels

Canonical paper: Human-level control through deep reinforcement learning (Mnih et al., 2015)

Reproduction target: Train DQN on ALE Breakout or ALE Pong through the Gymnasium or ALE interface.

Adaptation target: Move the method to MiniGrid MemoryEnv, a partially observed environment where the agent must remember an object seen early in the episode and act on that information later.

Required method change: Add explicit memory beyond a standard feed-forward network, for example recurrent DQN, gated state, or another well-justified memory mechanism.

Minimum success criteria

- Show that the baseline DQN learns a reasonable policy on Breakout or Pong.
- Evaluate the unmodified feed-forward version on MemoryEnv and document where it fails or plateaus.
- Implement a memory-augmented variant and compare it against the baseline on the new environment.
- Report at least 3 seeds and compare both learning behavior and final performance.

Official links

- DQN paper (Nature)
- ALE documentation - official Atari environment family
- ALE Breakout - recommended reproduction option
- ALE Pong - alternate reproduction option
- MiniGrid environment index
- MiniGrid MemoryEnv - adaptation target

Track 2: PPO

Canonical on-policy policy-gradient reinforcement learning

Canonical paper: Proximal Policy Optimization Algorithms (Schulman et al., 2017)

Reproduction target: Train PPO on Gymnasium MuJoCo HalfCheetah-v5 or Hopper-v5.

Adaptation target: Transfer PPO to MPE2 Simple Spread, where three agents must coordinate to cover landmarks while avoiding collisions.

Required method change: Extend single-agent PPO to a multi-agent setting using a principled design such as shared policy parameters, centralized critic, joint observation handling, or another clearly motivated multi-agent PPO variant.

Minimum success criteria

- Demonstrate stable learning on one MuJoCo locomotion benchmark.
- Run a straightforward PPO baseline on Simple Spread and document the coordination limits of the unmodified setup.
- Implement and evaluate a multi-agent extension on the same environment.
- Report at least 3 seeds and discuss how the adaptation addressed coordination, observation aggregation, and credit assignment.

Official links

- PPO paper (arXiv)
- Gymnasium MuJoCo overview - environment family used for reproduction
- MuJoCo documentation - simulator reference
- HalfCheetah-v5 - recommended reproduction option

- Hopper-v5 - alternate reproduction option
- MPE2 Simple Spread - current maintained home for the adaptation environment
- PettingZoo documentation - useful multi-agent API reference
- PettingZoo Simple Spread page - legacy page noting that simple_spread has moved to MPE2

Track 3: AlphaZero

Planning, self-play, and search-guided policy/value learning

Scope note: OpenSpiel's AlphaZero documentation explicitly describes the implementation as illustrative. It is suitable for games like Connect Four, but it is not designed to replicate superhuman Chess or Go training at the scale of the original paper.

Why this track matters: This track covers the search-and-self-play branch of classical RL. It is the most ambitious option and is best treated as a stretch project.

Canonical paper: Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm (Silver et al., 2017)

Reproduction target: Use OpenSpiel to build a course-scale AlphaZero-style pipeline on Connect Four rather than full chess or Go.

Adaptation target: Transfer the method to OpenSpiel Backgammon, which introduces stochasticity through explicit chance nodes.

Required method change: Modify the search or backup logic so that the method can handle chance events rather than assuming a purely deterministic perfect-information tree.

Minimum success criteria

- Show that the self-play and search pipeline improves over time on Connect Four.
- Explain clearly why a deterministic perfect-information implementation is mismatched to the new game.
- Implement a chance-aware adaptation and evaluate it on Backgammon or another instructor-approved chance-node game in OpenSpiel.
- Use win rate, match outcome, or another game-appropriate evaluation metric and compare the adapted pipeline against a simpler baseline.

Official links

- AlphaZero paper (arXiv)
- DeepMind AlphaZero overview - public overview article
- OpenSpiel introduction - general environment and algorithm library
- OpenSpiel AlphaZero docs - course-scale reference implementation
- OpenSpiel available games - includes Connect Four and Backgammon
- OpenSpiel chance-node reference - useful for the Backgammon adaptation

What does not count as an adaptation by itself: only retuning learning rates, changing the random seed, swapping frameworks, or lightly reshaping rewards without a broader argument about why the original method is mismatched to the new environment.

Environment index

Resource	Used for	Official page
Gymnasium	single-agent API and MuJoCo wrappers	Gymnasium docs
ALE	Atari reproduction for DQN	ALE docs
MiniGrid	DQN adaptation environment family	MiniGrid docs
MuJoCo	PPO reproduction environment family	MuJoCo docs; Gymnasium MuJoCo
PettingZoo	multi-agent API reference	PettingZoo docs
MPE2	current home of Simple Spread	MPE2 docs
OpenSpiel	AlphaZero reproduction and adaptation	OpenSpiel docs

Recommended milestones

- Week 1 - read the paper, set up the environment, and produce a minimal training script.
- Week 2 - obtain a first reproduction curve on the canonical task.
- Week 3 - run the unmodified method on the adaptation environment and identify the failure mode.
- Week 4 - implement the main adaptation, run ablations, and finalize the report and presentation.